

Fundamental Concepts in Data Processing

Nguyen Cong Duy

May 2025

Table of Contents

- 1 OLTP vs OLAP
- 2 ETL vs ELT
- 3 CAP theorem, ACID vs BASE
- 4 Lambda Architecture vs Kappa Architecture
- 5 Big Data Platforms

Purpose

- Handles day-to-day transactions (CRUD: Create, Read, Update, Delete)

Key Characteristics

- Real-time data, small transactions
- Optimized for speed and consistency
- Commonly used with Relational Databases (e.g., MySQL, PostgreSQL)

Examples

- Online shopping systems
- Banking systems
- User account management

Purpose

- Used for complex data analysis and business intelligence)

Key Characteristics

- Historical, aggregated data
- Optimized for complex queries and fast reading
- Usually built on Data Warehouses or Data Lakes

Examples

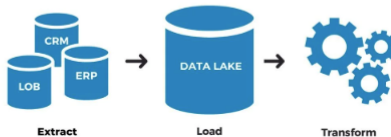
- Revenue analysis by quarter/year
- Customer behavior analysis
- BI dashboards and reporting tools

Criteria	OLTP	OLAP
Main Purpose	Handle small, fast transactions	Analyze large datasets, generate reports
Data Type	Real-time transactional data	Aggregated, historical data
Processing Speed	Very fast (Milliseconds)	Slower (Seconds to Minutes)
Workload	Many concurrent transactions	Few users, heavy queries
Database System	Relational DB (MySQL, PostgreSQL)	Data Warehouse (BigQuery, Redshift, Snowflake)
Typical Queries	CRUD (Create, Read, Update, Delete)	Aggregations (SUM, AVG, COUNT, GROUP BY)
Consistency Requirement	Very high (ACID compliant)	Lower priority, focus on data integrity
Use Cases	Banking, e-commerce, POS systems	Revenue reports, customer behavior analytics

ETL

VS.

ELT



In a distributed system, you can only guarantee two out of three properties:

- Consistency: All nodes see the same data at the same time
- Availability: Every request receives a (non-error) response
- Partition Tolerance: The system continues to operate despite network failures between nodes

ACID (SQL)

- **Atomicity:** A transaction must either be completed entirely or not at all.
- **Consistency:** Data must be valid before and after the transaction.
- **Isolation:** Transactions should not affect each other.
- **Durability:** Data remains permanent even in case of system failure.

BASE (NoSQL)

- **Basically Available:** The system responds even if the data are incomplete or an error occurs.
- **Soft-state:** Data may change without immediate consistency.
- **Eventually Consistent:** Data do not need to be consistent immediately, but will synchronize over time.

Lambda Architecture combines batch and real-time processing to handle massive data while ensuring fault tolerance and scalability. It consists of three layers:

- **Batch Layer:** Processes large data batches for accurate, historical insights.
- **Speed Layer:** Provides real-time data processing with lower accuracy.
- **Serving Layer:** Combines results from both layers for a comprehensive view.

While robust, its dual pipeline adds complexity and maintenance overhead.

Kappa Architecture simplifies data processing by focusing solely on real-time stream processing, unlike Lambda Architecture that separates batch and real-time processing.

Key features of Kappa Architecture:

- **Single Processing Pipeline:** All data is processed as a real-time stream, eliminating the need for a separate batch layer.
- **Reprocessing Capability:** Historical data can be reprocessed by replaying the same stream through the processing engine, avoiding the need for a separate batch system.
- **Simplicity:** Kappa offers lower complexity and faster insights compared to Lambda Architecture, but may not be suitable for highly accurate batch processing.

Kappa is ideal for systems that rely heavily on real-time data, offering faster insights and easier maintenance.

Lambda Architecture vs Kappa Architecture

Aspect	Lambda Architecture	Kappa Architecture
Processing Model	Combines batch and real-time processing.	Focuses solely on stream (real-time) processing.
Layers	Batch layer, Speed layer, Serving layer.	Single pipeline for both real-time and historical data.
Complexity	Higher due to managing separate batch and speed layers.	Simpler, as it uses only one stream processing layer.
Fault Tolerance	Fault-tolerant, as batch processing ensures data accuracy.	Fault-tolerant with real-time processing but depends on stream integrity.
Use Case	Suitable for both real-time and batch processing needs.	Best for real-time processing; batch processing is less emphasized.
Data Reprocessing	Batch layer allows accurate reprocessing of historical data.	Reprocessing is done by replaying the stream in real-time.
Latency	Higher latency in batch processing; low latency in real-time.	Low latency for all data due to stream processing.
Accuracy	Batch layer provides high accuracy, speed layer offers immediate but less accurate results.	Provides consistent results, but may not match the accuracy of dedicated batch processing.

Logical Architecture

- **HDFS (Hadoop Distributed File System):** A distributed file system that stores large data sets across multiple machines.
- **MapReduce:** A programming model for processing and generating large datasets by splitting tasks into smaller jobs.
- **YARN (Yet Another Resource Negotiator):** Manages resources and schedules tasks for the cluster.

Physical Architecture

- **NameNode:** Stores the metadata for HDFS, tracks file locations and directories.
- **DataNodes:** Store the actual data blocks in the HDFS.
- **ResourceManager:** Manages resources and job scheduling for MapReduce.

Hadoop 3 brings significant enhancements over Hadoop 2, particularly in scalability, storage efficiency, and resource management - most notably through YARN Federation.

- Hadoop 2: Only one ResourceManager per cluster → becomes a bottleneck at large scale.
- Hadoop 3: Supports multiple ResourceManagers via YARN Federation → enables scalability and high availability.

Logical Architecture

- **Driver Program:** Coordinates the work and manages the SparkContext, which communicates with the cluster.
- **Cluster Manager:** Can be YARN, Mesos, or Spark's standalone cluster manager that manages resources and job execution.
- **Executors:** Run tasks and store data in a Spark cluster, handle the execution of jobs.

Physical Architecture

- **Worker Nodes:** Perform the computation in Spark, execute tasks and store data in memory.
- **RDD (Resilient Distributed Dataset):** A fundamental data structure in Spark, representing distributed collections of objects.
- **SparkContext:** Initializes and coordinates jobs, connects to the cluster, and manages execution.

Logical Architecture

- **Producers:** Send data to Kafka topics.
- **Consumers:** Subscribe to topics and read data.
- **Brokers:** Handle the storage, distribution, and transmission of messages between producers and consumers.

Physical Architecture

- **Kafka Cluster:** A collection of Kafka brokers responsible for message storage and processing.
- **Partitions:** Each topic is split into partitions, allowing parallel processing and fault tolerance.
- **ZooKeeper:** Coordinates and manages the Kafka brokers, ensuring fault tolerance and consensus across the cluster.

Kafka 4 introduces major architectural and performance improvements over Kafka 3, focusing on scalability, resilience, and simplicity - especially with the removal of ZooKeeper.

- Kafka 3: Still relies on ZooKeeper for metadata management and controller coordination.
- Kafka 4: Completely removes ZooKeeper - introduces KRaft (Kafka Raft) for a self-managed, simpler architecture.